Universidad
Nacional del
Altiplano de Puno

UNIVERSIDAD NACIONAL DEL ALTIPLANO
FACULTAD DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA

EJERCICIOS DE PILAS (STACK)

Práctica Introductoria – Principio LIFO y Operaciones Básicas

ESTRUCTURA DE DATOS

Docente: Fred Torres Cruz
Estudiante: Blays Kevin Mamani Quispe
Link de Repositorio GitHub:
https://github.com/blayskevinmamaniquispe-lab/Estructura-de-Datos/tree/main/Ejerccicios

Indicaciones Generales:

- Resuelva los ejercicios utilizando la estructura `std::stack` de C++.
- Identifique en todo momento el elemento ubicado en el **TOPE**.
- Observe y compruebe el comportamiento **LIFO** (*Last In, First Out*).

Puno, Perú
2026

Índice

Introducción: El Principio LIFO y Plantilla Base 2

Ejercicio 1. Apilar números enteros 3

Ejercicio 2. Invertir una palabra 4

Ejercicio 3. Verificar paréntesis balanceados 5

Ejercicio 4. Historial de acciones – Deshacer (Undo) 7

Ejercicio 5. Conversión de decimal a binario 9

Pregunta de Cierre y Análisis 10

Introducción: El Principio LIFO y Plantilla Base

En una estructura de datos tipo **Pila (Stack)**, los elementos se agregan y remueven siguiendo el principio **LIFO** (*Last In, First Out* – Último en entrar, primero en salir).

TOPE $\rightarrow$ **30** 20 10

Idea clave: si se insertaron 10, 20 y 30, el primero en salir será 30.

Plantilla Base en C++

El siguiente código constituye la estructura mínima con `std::stack`:

PLANTILLA BASE EN C++

```cpp
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> pila;

    pila.push(10);
    pila.push(20);
    pila.push(30);

    while (!pila.empty()) {
        cout << pila.top() << " ";
        pila.pop();
    }

    cout << endl;
    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ g++ plantilla.cpp -o plantilla
$ ./plantilla
30 20 10
```

Ejercicio 1. Apilar números enteros

Enunciado: Crear un programa que permita ingresar 5 números enteros y almacenarlos en una pila. Luego, retirar y mostrar los elementos uno por uno hasta dejar la pila vacía. El orden de salida debe permitir comprobar el funcionamiento LIFO.

Condición: Use las operaciones `push()`, `top()`, `pop()` y `empty()`.

CÓDIGO FUENTE: Ejercicio 1

```cpp
#include <iostream>
#include <stack>

using namespace std;

int main() {
    stack<int> pila;
    int numero;

    cout << "--- INGRESO DE 5 N MEROS ENTEROS ---\n";
    for (int i = 1; i <= 5; i++) {
        cout << "Ingrese el numero " << i << ": ";
        cin >> numero;
        pila.push(numero);
    }

    cout << "\n--- DESAPILANDO ELEMENTOS (ORDEN LIFO) ---\n";
    while (!pila.empty()) {
        cout << "Elemento en el TOPE: " << pila.top() << endl;
        pila.pop();
    }

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ g++ ejercicio1.cpp -o ejercicio1
$ ./ejercicio1
--- INGRESO DE 5 N MEROS ENTEROS ---
Ingrese el numero 1: 4
Ingrese el numero 2: 8
Ingrese el numero 3: 12
Ingrese el numero 4: 16
Ingrese el numero 5: 20

--- DESAPILANDO ELEMENTOS (ORDEN LIFO) ---
Elemento en el TOPE: 20
Elemento en el TOPE: 16
Elemento en el TOPE: 12
Elemento en el TOPE: 8
Elemento en el TOPE: 4
```

Ejercicio 2. Invertir una palabra

Enunciado: Leer una palabra ingresada por el usuario. Insertar cada carácter en una pila y, posteriormente, desapilar los caracteres para construir y mostrar la palabra en orden inverso.

Condición: Analice por qué una pila permite invertir naturalmente el orden de los elementos.

CÓDIGO FUENTE: Ejercicio 2

```cpp
#include <iostream>
#include <stack>
#include <string>

using namespace std;

int main() {
    stack<char> pilaCaracteres;
    string palabra;

    cout << "Ingrese una palabra: ";
    cin >> palabra;

    // Apilar cada car cter de la palabra
    for (char c : palabra) {
        pilaCaracteres.push(c);
    }

    string palabraInvertida = "";

    // Desapilar caracteres para revertir el orden
    while (!pilaCaracteres.empty()) {
        palabraInvertida += pilaCaracteres.top();
        pilaCaracteres.pop();
    }

    cout << "\nPalabra original: " << palabra << endl;
    cout << "Palabra invertida: " << palabraInvertida << endl;

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ g++ ejercicio2.cpp -o ejercicio2
$ ./ejercicio2
Ingrese una palabra: DATOS

Palabra original: DATOS
Palabra invertida: SOTAD
```

Ejercicio 3. Verificar paréntesis balanceados

Enunciado: Crear un programa que determine si una expresión tiene sus paréntesis correctamente balanceados. Cada vez que aparezca un paréntesis de apertura `(`, se almacena en la pila. Cuando aparezca `)`, se comprueba que exista una apertura pendiente.

Condición: La expresión será correcta únicamente si nunca se intenta cerrar un paréntesis inexistente y la pila queda vacía al finalizar.

CÓDIGO FUENTE: Ejercicio 3

```cpp
#include <iostream>
#include <stack>
#include <string>

using namespace std;

bool estaBalanceado(const string& expresion) {
    stack<char> pila;

    for (char c : expresion) {
        if (c == '(') {
            pila.push(c);
        } else if (c == ')') {
            if (pila.empty()) {
                return false; // Intento de cerrar par ntesis sin apertura
            }
            pila.pop();
        }
    }

    return pila.empty(); // Es correcto si no quedan aperturas pendientes
}

int main() {
    string expresion;

    cout << "Ingrese la expresion matematica: ";
    cin >> expresion;

    if (estaBalanceado(expresion)) {
        cout << "Resultado: La expresion esta CORRECTAMENTE balanceada." <<
            endl;
    } else {
        cout << "Resultado: La expresion esta INCORRECTAMENTE balanceada."
           << endl;
    }

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ g++ ejercicio3.cpp -o ejercicio3
$ ./ejercicio3
Ingrese la expresion matematica: (a+b)*(c-d)
Resultado: La expresion esta CORRECTAMENTE balanceada.

$ ./ejercicio3
```

```
Ingrese la expresion matematica: ((a+b)
Resultado: La expresion esta INCORRECTAMENTE balanceada.
```

Ejercicio 4. Historial de acciones – Deshacer (Undo)

Enunciado: Simular la función Deshacer de un editor. Cada acción realizada por el usuario se almacena en una pila. Cuando se solicite deshacer, el programa debe mostrar y eliminar la última acción registrada.

Condición: Incluya una opción para mostrar cuál es la acción actualmente ubicada en el tope de la pila.

CÓDIGO FUENTE: Ejercicio 4

```cpp
#include <iostream>
#include <stack>
#include <string>

using namespace std;

int main() {
    stack<string> historial;
    int opcion;
    string accion;

    do {
        cout << "\n--- EDITOR DE TEXTO (DESHACER) ---\n";
        cout << "1. Realizar nueva accion\n";
        cout << "2. Ver accion en el TOPE\n";
        cout << "3. Deshacer (Undo)\n";
        cout << "4. Salir\n";
        cout << "Seleccione una opcion: ";
        cin >> opcion;
        cin.ignore();

        switch (opcion) {
            case 1:
                cout << "Ingrese la accion realizada: ";
                getline(cin, accion);
                historial.push(accion);
                cout << "Accion registrada con exito.\n";
                break;
            case 2:
                if (!historial.empty()) {
                    cout << "TOPE actual: \"" << historial.top() << "\"\n";
                } else {
                    cout << "El historial esta vacio.\n";
                }
                break;
            case 3:
                if (!historial.empty()) {
                    cout << "Deshaciendo accion: \"" << historial.top() <<
                        "\"\n";
                    historial.pop();
                } else {
                    cout << "No hay acciones para deshacer.\n";
                }
                break;
            case 4:
                cout << "Saliendo del simulador...\n";
                break;
            default:
                cout << "Opcion no valida.\n";
```

```
            }
        } while (opcion != 4);

        return 0;
    }
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ g++ ejercicio4.cpp -o ejercicio4
$ ./ejercicio4
--- EDITOR DE TEXTO (DESHACER) ---
1. Realizar nueva accion
2. Ver accion en el TOPE
3. Deshacer (Undo)
4. Salir
Seleccione una opcion: 1
Ingrese la accion realizada: escribir texto

Seleccione una opcion: 1
Ingrese la accion realizada: eliminar palabra

Seleccione una opcion: 2
TOPE actual: "eliminar palabra"

Seleccione una opcion: 3
Deshaciendo accion: "eliminar palabra"
```

Ejercicio 5. Conversión de decimal a binario

Enunciado: Leer un número entero decimal positivo y convertirlo a binario dividiendo repetidamente entre 2 y almacenando el residuo en la pila. Finalmente, desapilar los residuos para obtener el número en el orden correcto.

Condición: No utilice funciones directas de conversión a binario; la solución debe evidenciar el uso de la pila.

CÓDIGO FUENTE: Ejercicio 5

```cpp
#include <iostream>
#include <stack>

using namespace std;

int main() {
    stack<int> residuos;
    int numeroDecimal;

    cout << "Ingrese un numero entero decimal positivo: ";
    cin >> numeroDecimal;

    if (numeroDecimal < 0) {
        cout << "Por favor ingrese un numero positivo." << endl;
        return 1;
    }

    if (numeroDecimal == 0) {
        cout << "Binario: 0" << endl;
        return 0;
    }

    int aux = numeroDecimal;
    while (aux > 0) {
        residuos.push(aux % 2);
        aux /= 2;
    }

    cout << "Numero decimal: " << numeroDecimal << endl;
    cout << "Representacion en binario: ";
    while (!residuos.empty()) {
        cout << residuos.top();
        residuos.pop();
    }
    cout << endl;

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ g++ ejercicio5.cpp -o ejercicio5
$ ./ejercicio5
Ingrese un numero entero decimal positivo: 13
Numero decimal: 13
Representacion en binario: 1101
```

Pregunta de Cierre y Análisis

¿Qué característica común tienen los cinco problemas que hace apropiado utilizar una pila?

Respuesta y Justificación Técnica:

La característica común fundamental es el **requerimiento de un procesamiento en orden inverso al de llegada**, donde el **último elemento procesado o ingresado es el primero en ser atendido o recuperado** (Principio **LIFO** – *Last In, First Out*).

1. **Inversión de Secuencias (Ejercicios 1 y 2):** Para desapilar números o una palabra de forma invertida, el carácter/número ingresado en último lugar debe salir primero.

2. **Anidamiento y Pareo de Simetría (Ejercicio 3):** Al validar paréntesis, el último paréntesis de apertura encontrado es siempre el primero que debe cerrarse con uno de cierre.

3. **Deshacer Operaciones en Cascada (Ejercicio 4):** En un historial de acciones, la última acción realizada por el usuario es siempre la primera que se debe revertir.

4. **Reconstrucción del Orden Correcto (Ejercicio 5):** Los residuos binarios se generan del bit menos significativo (LSB) al más significativo (MSB); desapilarlos permite imprimir el número en la lectura correcta de izquierda a derecha.